



LECTURE 3

DEEP LEARNING I

What is Learning?

PRESS → OR SPACE. →

© Marc Rußwurm

Licensed under [Creative Commons Attribution-NonCommercial 4.0 International \(CC BY-NC 4.0\)](#).

You may share and adapt these slides for teaching, research, and other non-commercial educational purposes with attribution.

Commercial training, paid workshops, consulting seminars, or incorporation into commercial course products require prior permission.

In this lecture

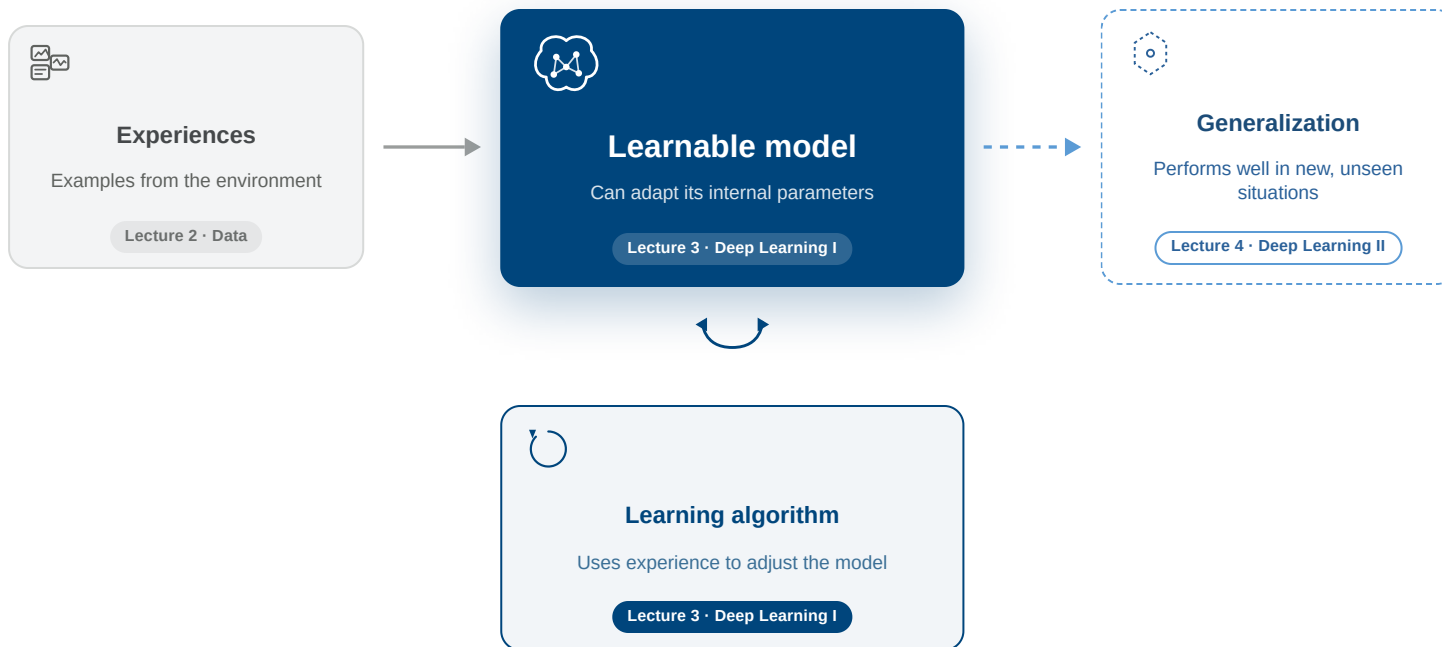
- Inputs, targets, and supervised learning
- Classification and regression
- Train / validation / test splits
- Decision trees and ensemble methods
- Random forests as the main model

The Aspects of Learning



via [GIPHY](#), original source [imgur.com/gallery/FBMm2DO](#)

What Does a System Need to Learn?



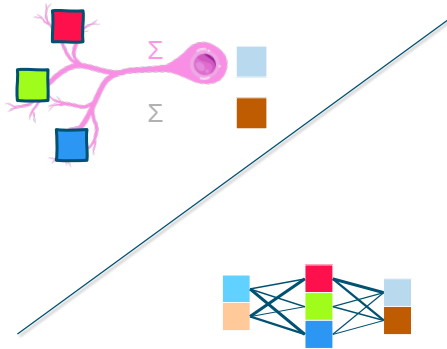
INGREDIENTS FOR LEARNING

SUCCESS CRITERION

Two Core Topics in This Lecture

DEEP LEARNING MODEL

1. **Architectures** — MLP, CNN, Transformer
2. **Tasks** — Classification & Segmentation



MODEL TRAINING

1. **Loss surfaces** and gradient descent
2. Forward pass → Loss → Backprop → Update

Minimize Error/Loss function

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \mathcal{L}(f_{\mathbf{W}}(\mathbf{x}), \mathbf{y})$$

Through Iterative Gradient Descent:

$$\mathbf{W}^{(t+1)} = \mathbf{W}^t - \nabla \mathcal{L}(f_{\mathbf{W}}, \mathcal{D}_{\text{train}})$$

more complex models have more complex loss surfaces

Data and Experience



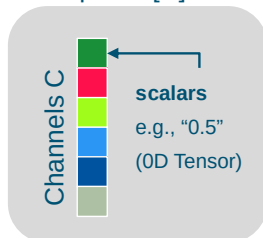
Data is the window through which a learner experiences the world.

Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding Machine Learning: From Theory to Algorithms*.

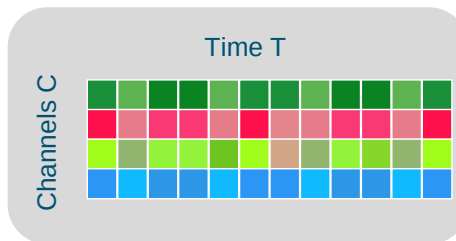
Photo by [Dương Nhân](#) on [Pexels](#)

Data as blocks of numbers - E.g., Tensors

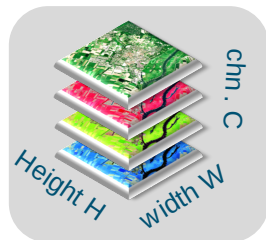
1D Tensor: Vector
e.g., pixels or
class probs [C]



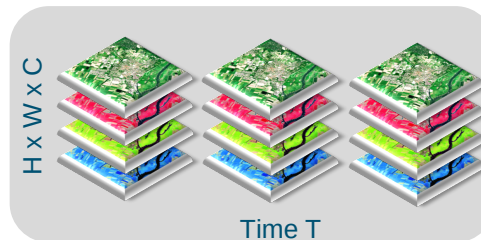
2D Tensor: Matrix
Time Series [T x C]



3D Tensors
Images [H x W x C]



4D Tensors
Img. Time Series [T x H x W x C]



Data samples from a distribution

The Learnable Model



Photo by [Vitaly Gariev](#) on [Pexels](#)

Our Learned Understanding

CHINESE

□□□□□□□□□□□□□□

ARABIC

لقد تعلّم عقلنا أن يفهم هذه الجملة.

HEBREW

המוח שלנו למד להבין את המשפט הזה.

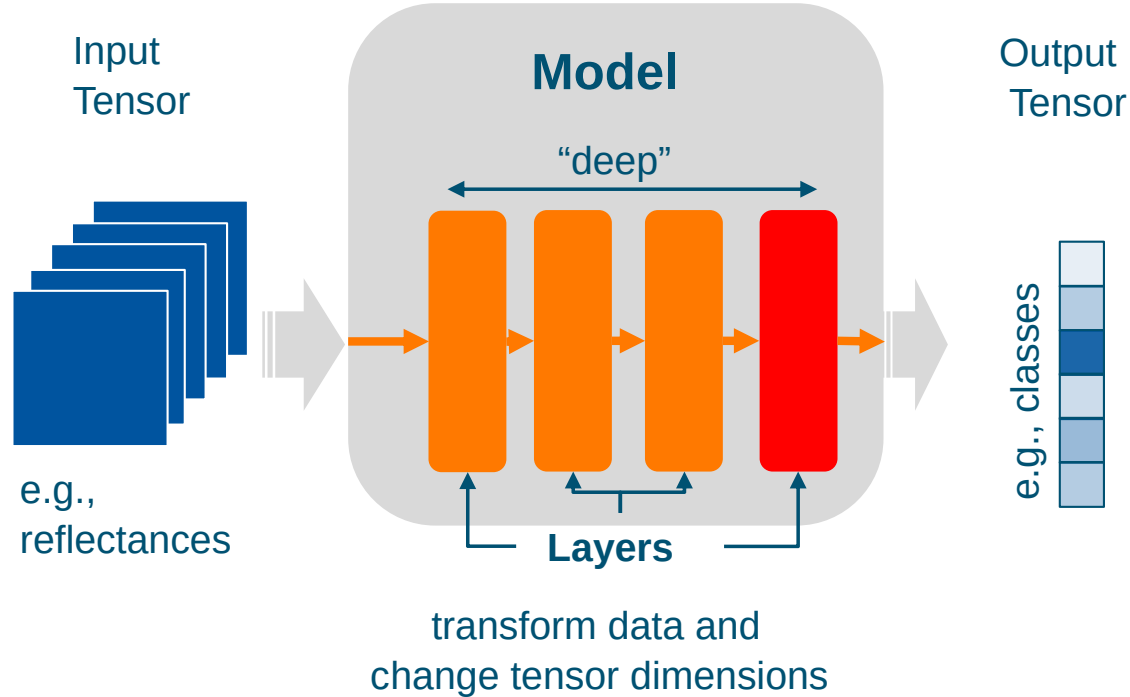
KOREAN

□□□ □□□ □ □□□ □□□□ □□ □□□□□.

LATIN

Our mind has learned to understand this sentence.

Deep Learning Model



Sentinel-2 as an Image Tensor

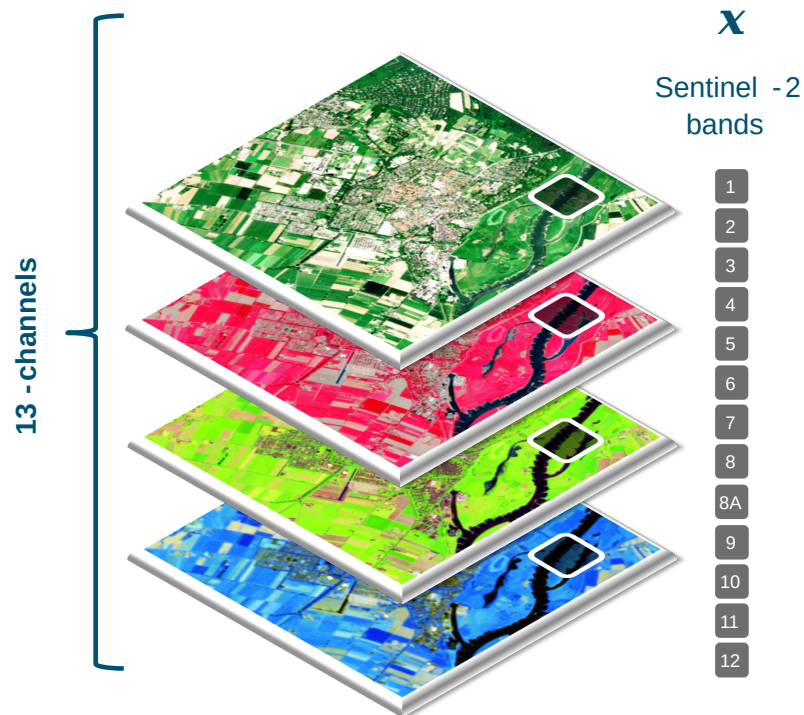


Image Classification

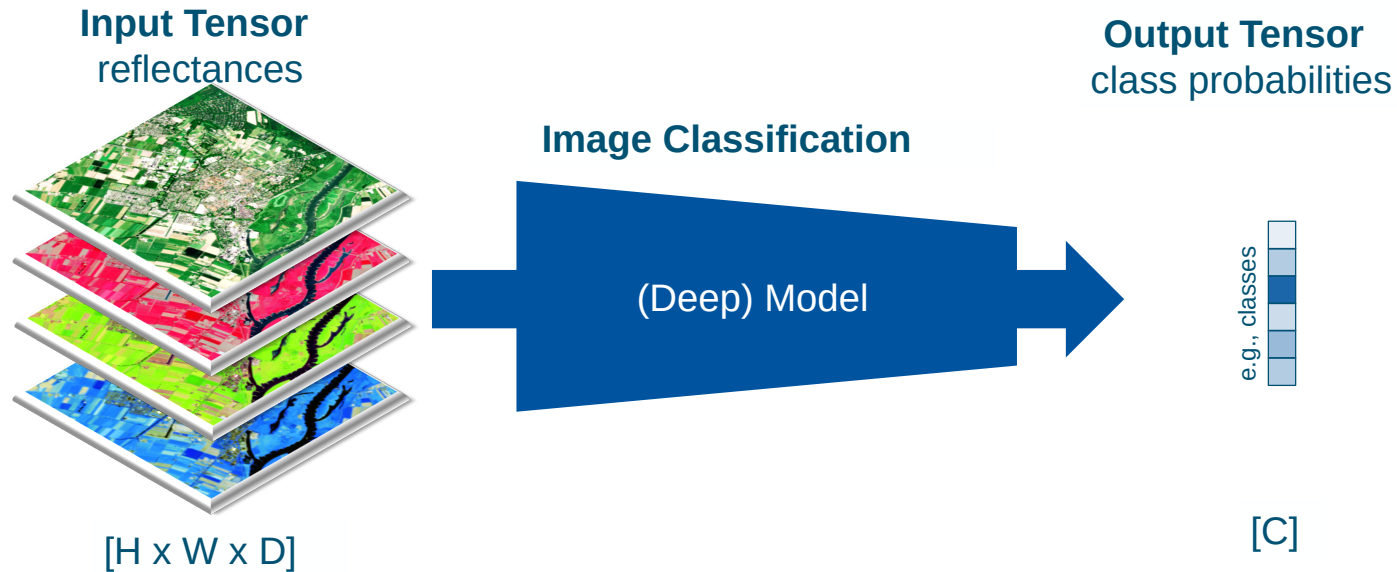
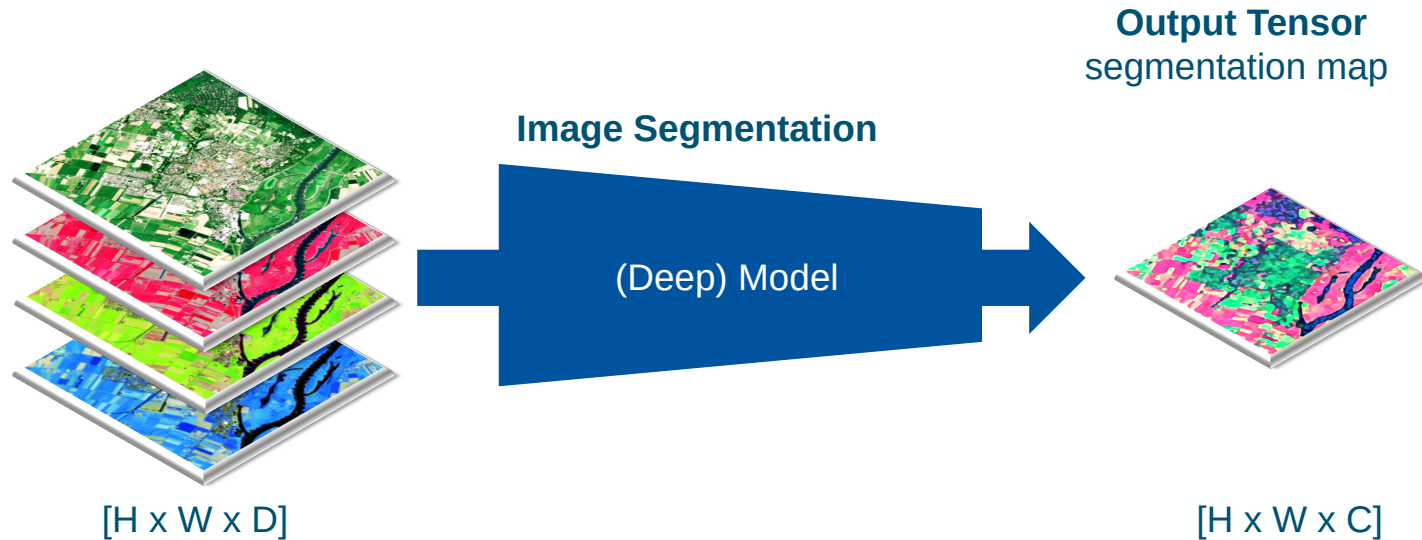


Image Segmentation



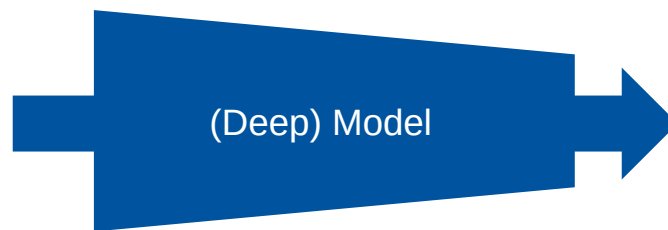
Object Detection

Input Tensor
reflectances



$[H \times W \times 3]$

Object Detection

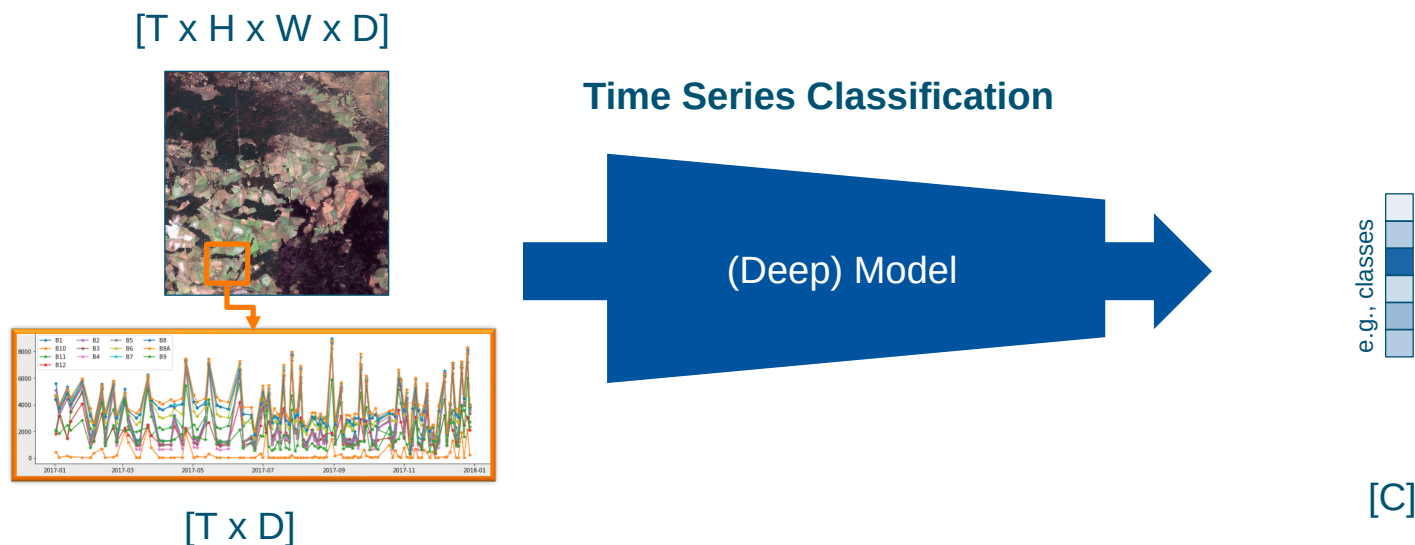


Output Tensor
bounding boxes

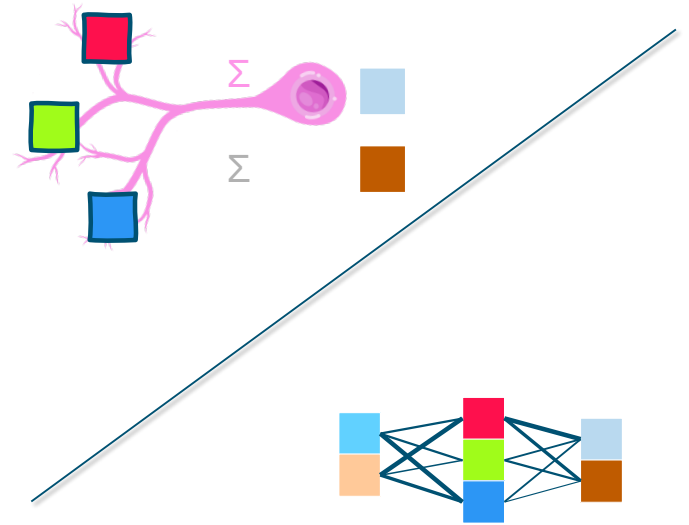
X	Y	H	W	
Green	Green	Green	Green	Bbox 1
Red	Red	Red	Red	Bbox 2
Yellow	Yellow	Yellow	Yellow	Bbox 3
Blue	Blue	Blue	Blue	...

$[N \times 4]$

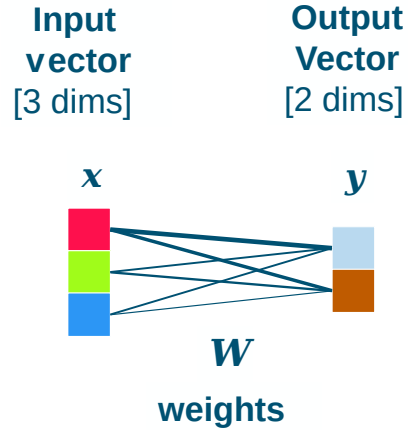
Time Series Classification



The Multi-Layer Perceptron (MLP) Model

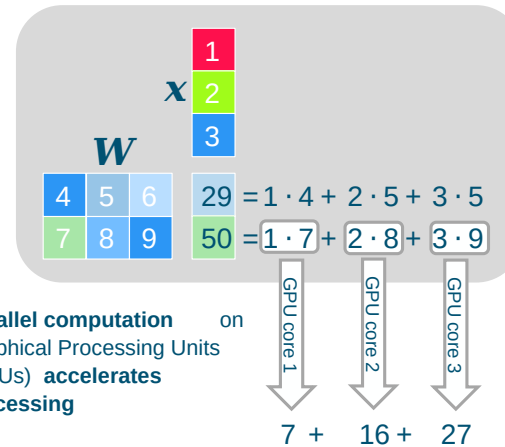


Linear Transformation



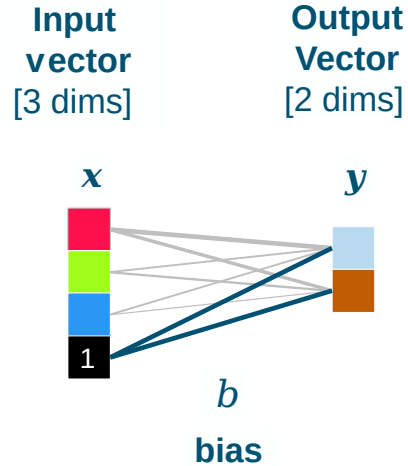
Linear Transformation

$$y = Wx = \sum w_i x_i$$



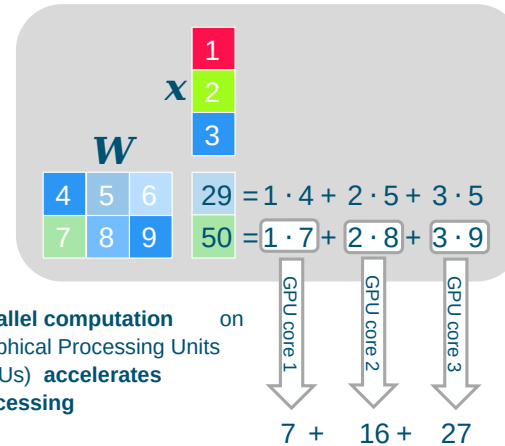
Parallel computation on Graphical Processing Units (GPUs) accelerates processing

Linear Transformation with a Bias Term



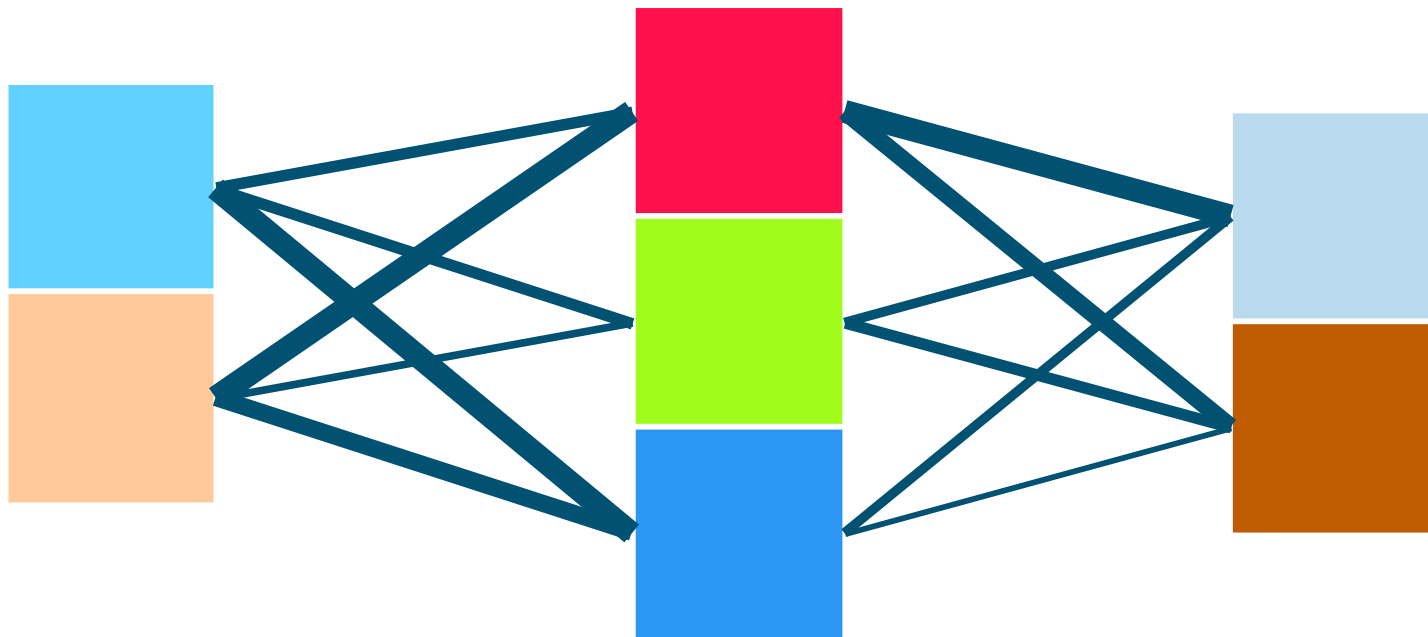
Linear Transformation

$$y = Wx = \sum w_i x_i + b$$

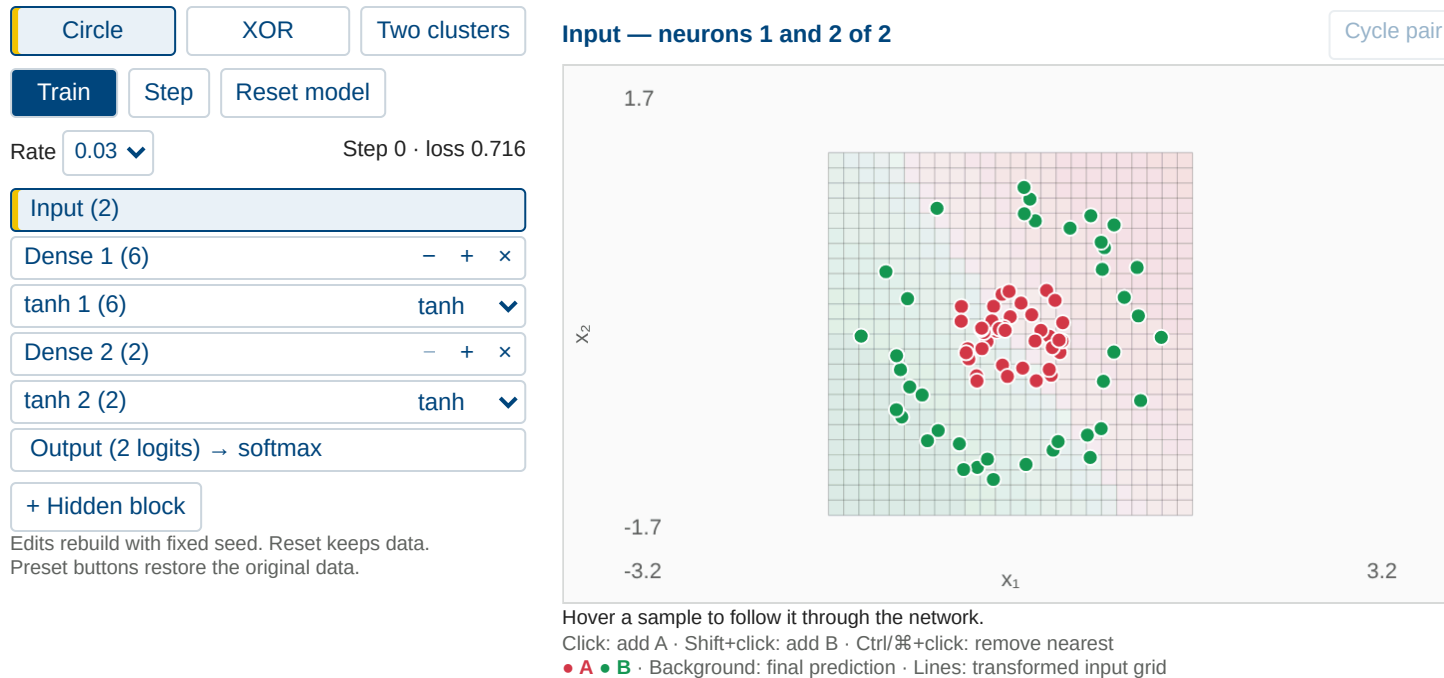


Parallel computation on
Graphical Processing Units
(GPUs) accelerates
processing

Weighted Connections

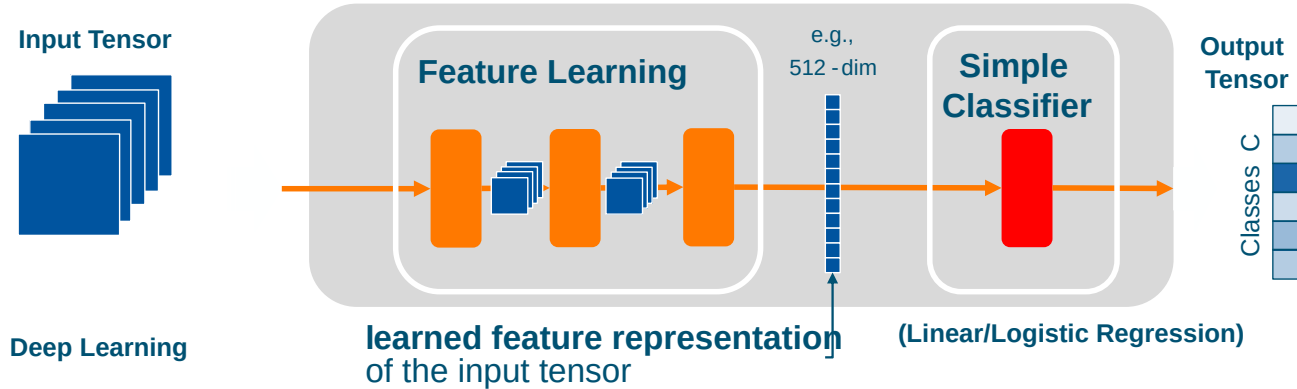


How an MLP transforms feature space



Inspired by Andrej Karpathy's [ConvNetJS Classify2D](#) demo. Implemented with Claude and Codex.

Machine Learning and Deep Learning



Machine Learning

Input Tensor



Design knowledge in **designing** features mathematically

Features Design

(e.g., NDVI, NDWI, GLCM Textures)

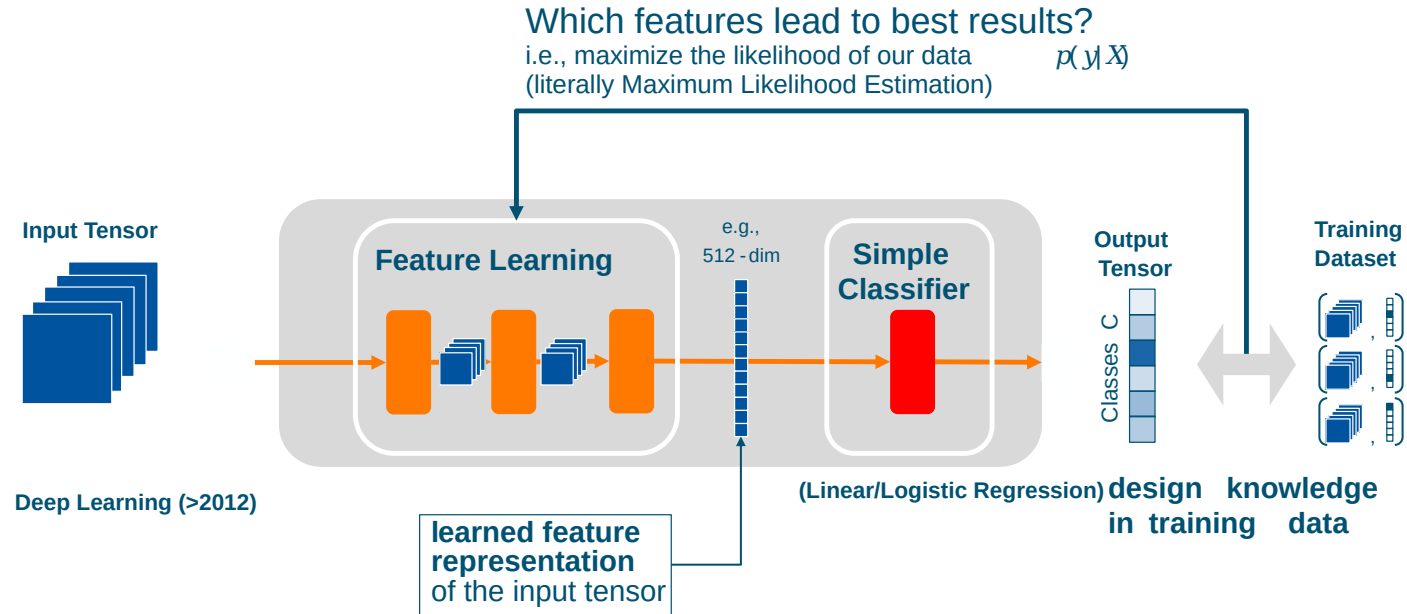
Complex Classifier

(Random Forest, Boosting)

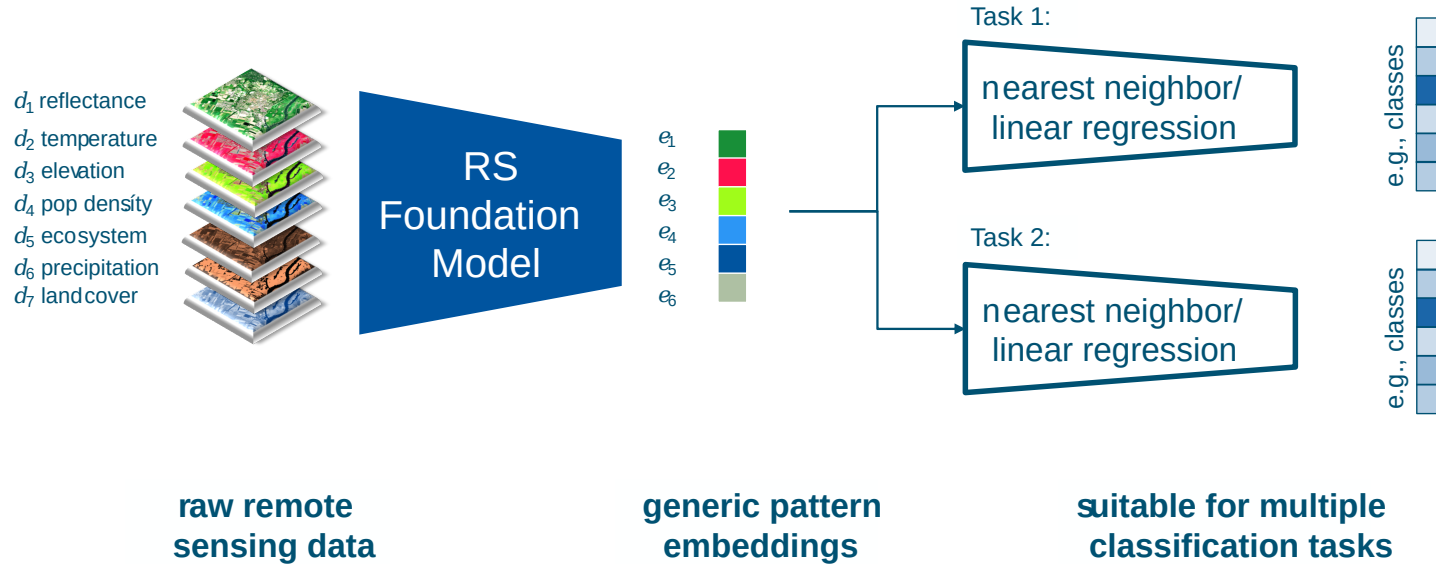
Output Tensor



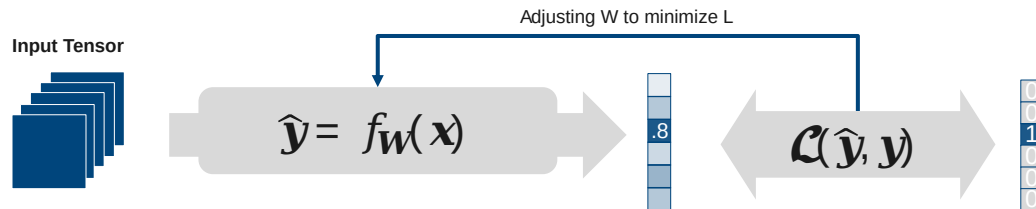
Feature Learning



Foundation Models



Training: Adjusting Weights to Minimize Loss



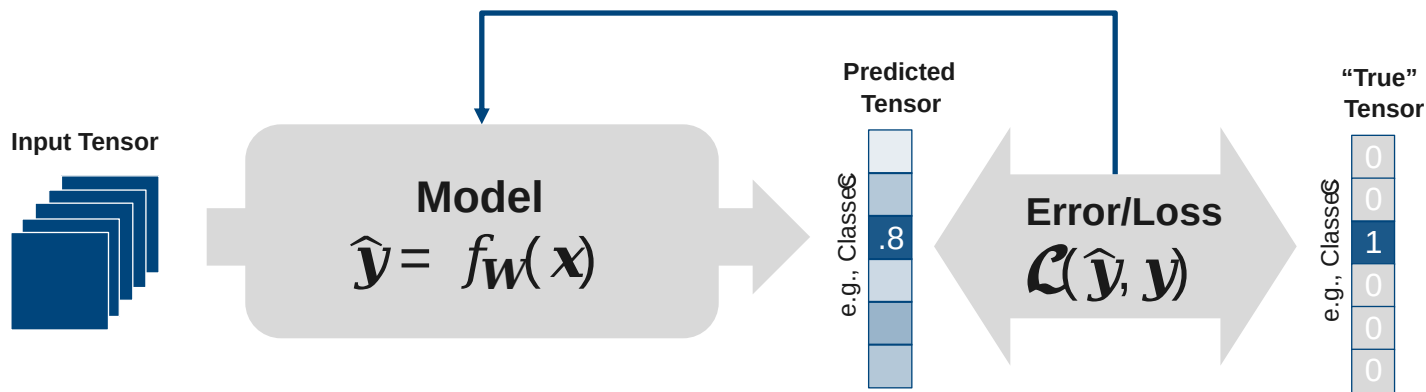
$$\mathcal{L}(\mathcal{D}_{\text{train}}) = \sum_{\mathbf{x}, \mathbf{y} \in \mathcal{D}_{\text{train}}} \mathcal{L}(f(\mathbf{x}), \mathbf{y})$$



The Learning Objective: argmin

Minimize Error/Loss function

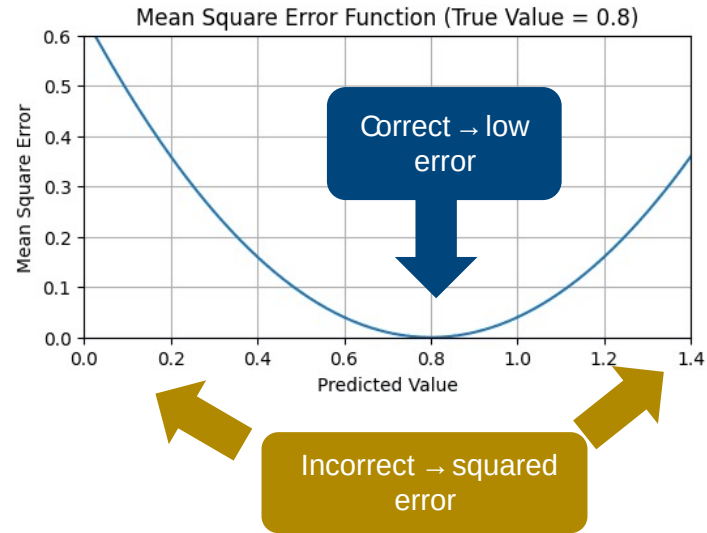
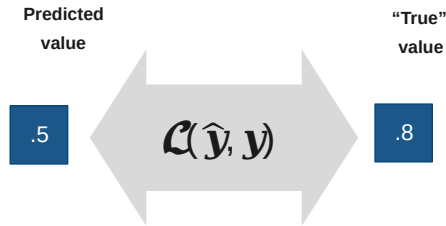
$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \mathcal{L}(f(\mathbf{x}), \mathbf{y})$$



Loss Function: Mean Squared Error

Regression: Mean Squared Error

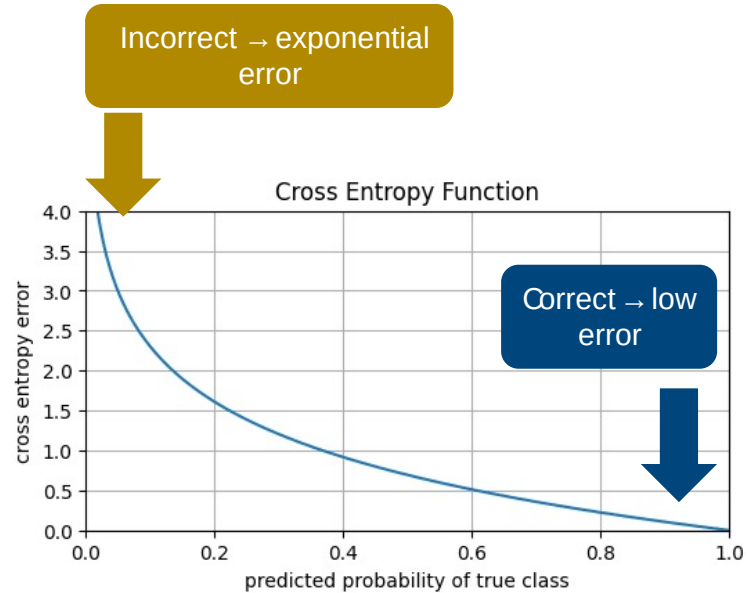
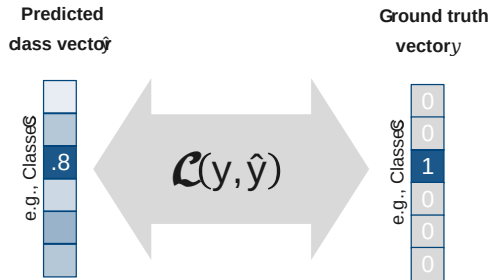
$$\mathcal{L}(y, \hat{y}) = (y - \hat{y})^2$$



Loss Function: Cross-Entropy

Classification: Cross Entropy

$$\mathcal{L}(y, \hat{y}) = \sum_{c \in \text{Classes}} -y_c \log(\hat{y}_c)$$



Loss Surfaces

Minimize Error/Loss function

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \mathcal{L}(f_{\mathbf{W}}(\mathbf{x}), \mathbf{y})$$

Through Iterative Gradient Descent:

$$\mathbf{W}^{(i+1)} = \mathbf{W}^i - \nabla \mathcal{L}(f_{\mathbf{W}}, \mathcal{D}_{\text{train}})$$

more complex models have more complex loss surfaces

Gradient Descent

Minimize Error/Loss function

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \mathcal{L}(f_{\mathbf{W}}(\mathbf{x}), \mathbf{y})$$

Through Iterative Gradient Descent:

$$\mathbf{W}^{(i+1)} = \mathbf{W}^i - \nabla \mathcal{L}(f_{\mathbf{W}}, \mathcal{D}_{\text{train}})$$

new weights

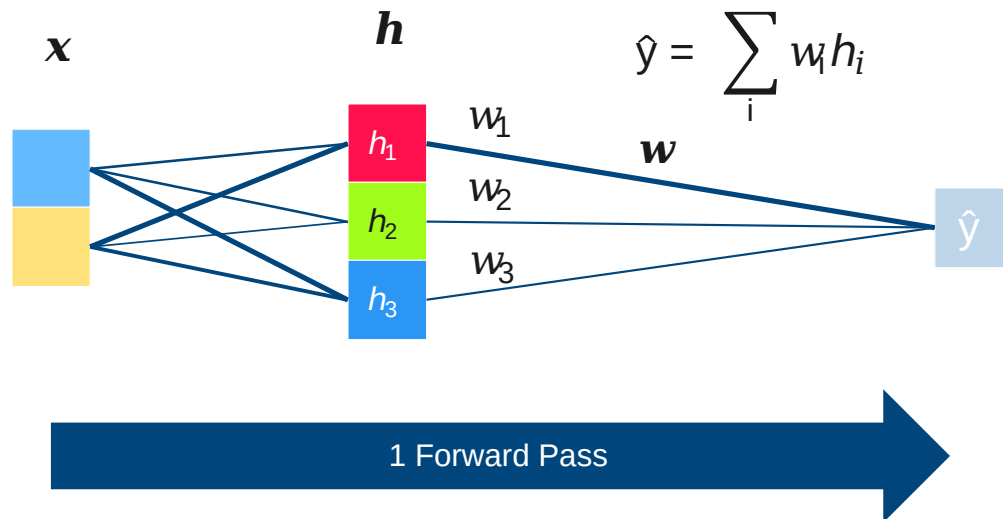
old weights



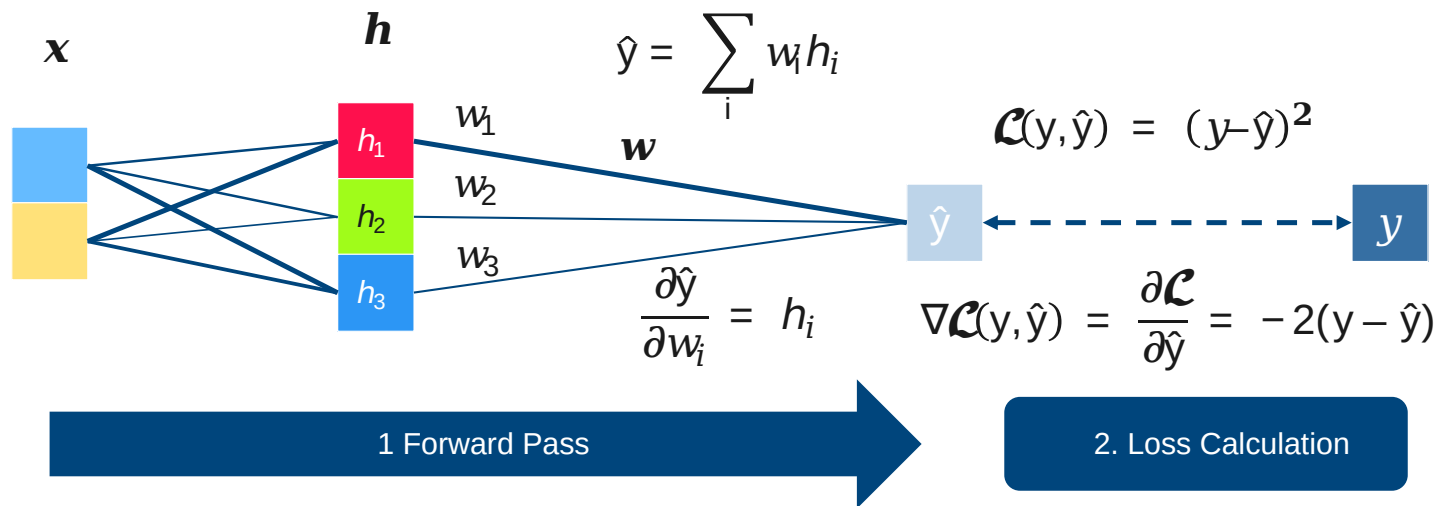
Gradients of the Loss function over a “batch” of training data $\mathcal{D}_{\text{train}}$

“What weights change the loss the most?”

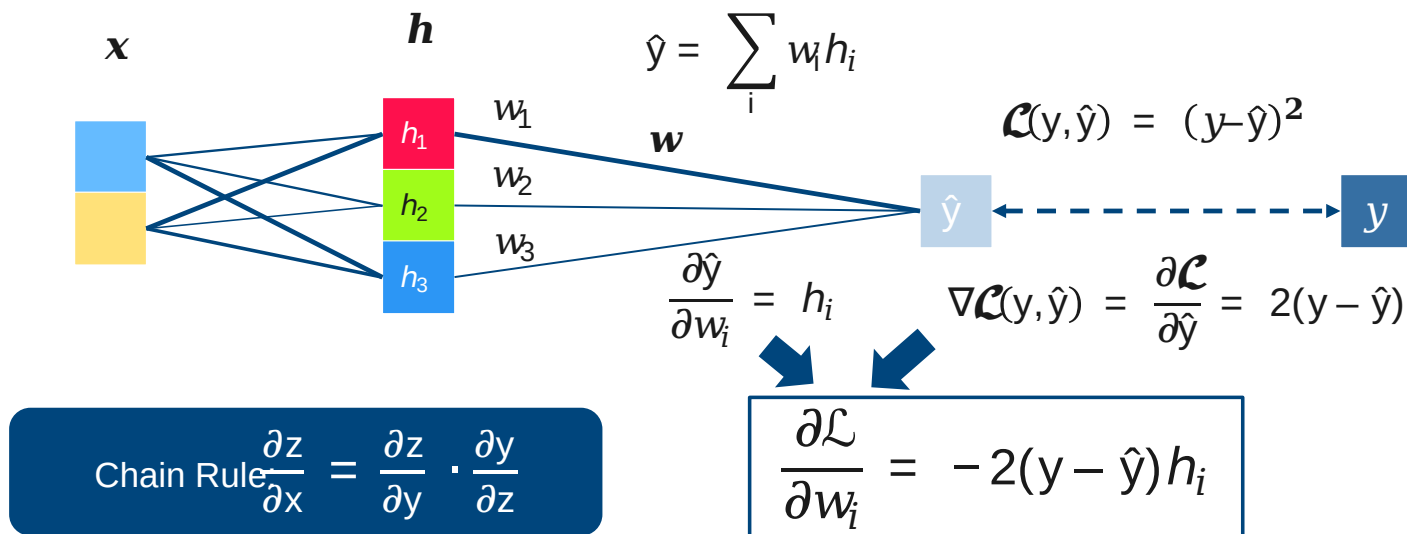
Backpropagation



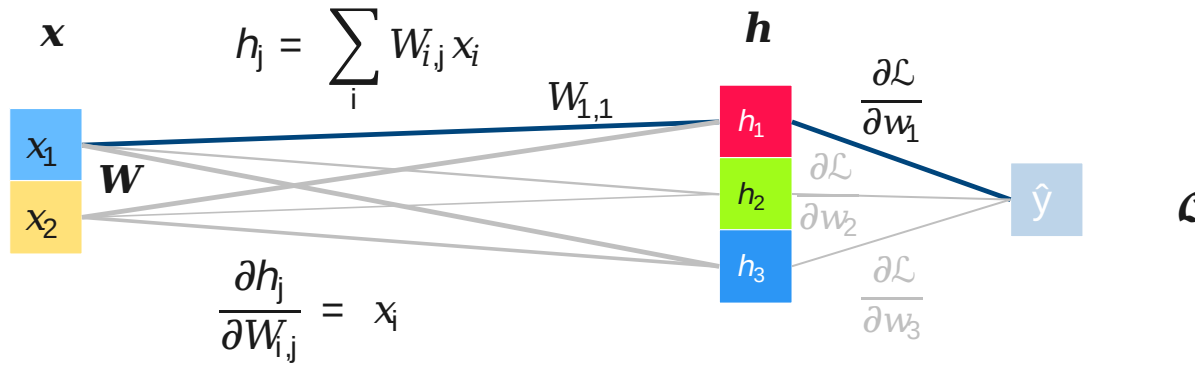
Backpropagation



Backpropagation

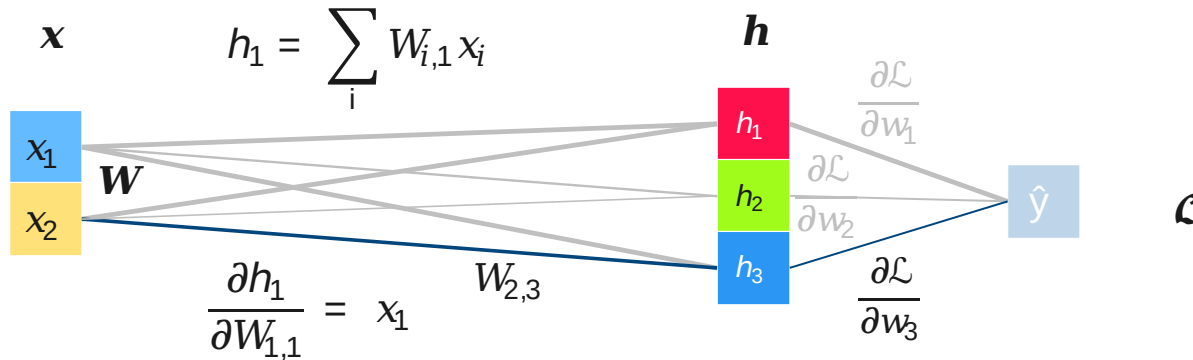


Backpropagation



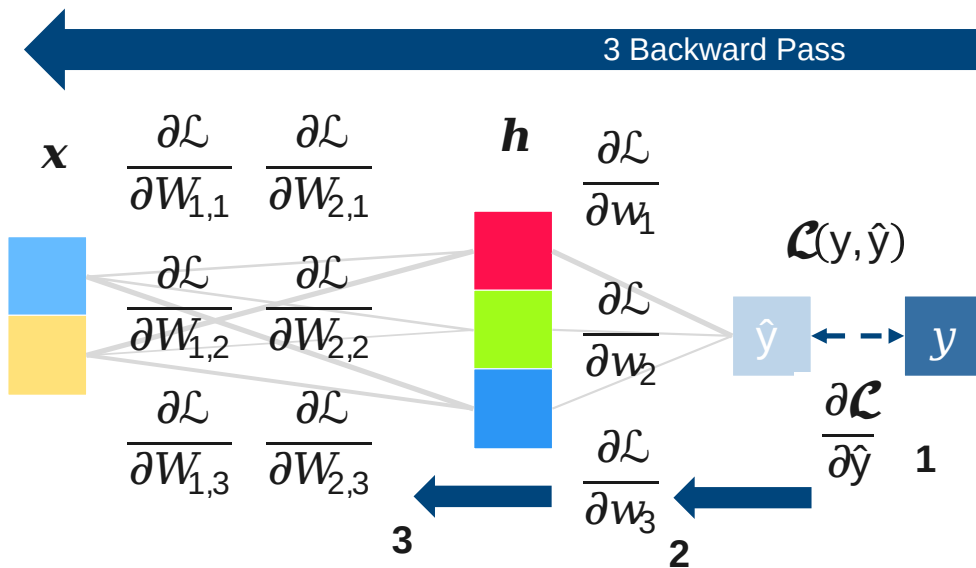
$$\frac{\partial \mathcal{L}}{\partial W_{1,1}} = x_1 \frac{\partial \mathcal{L}}{\partial w_1} = -x_1 2(y - \hat{y}) h_1$$

Backpropagation



$$\frac{\partial \mathcal{L}}{\partial W_{2,3}} = x_2 \frac{\partial \mathcal{L}}{\partial w_3} = -x_2 2(y - \hat{y}) h_3$$

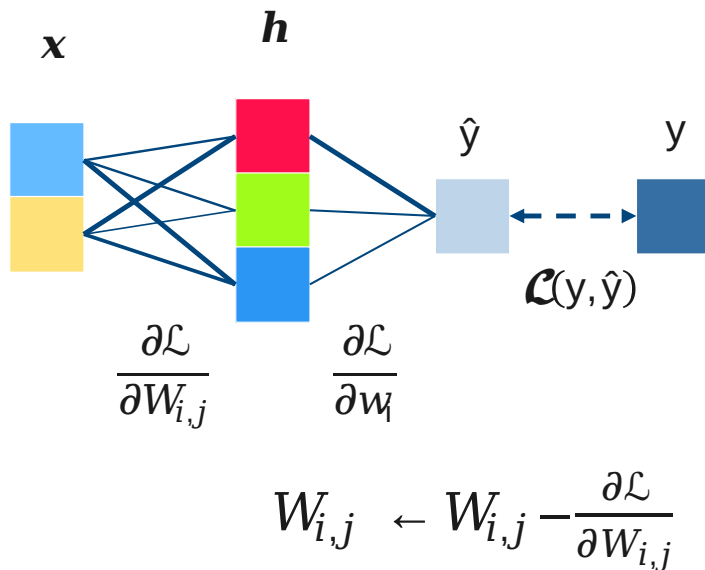
Backpropagation



Save computation:
Gradients of earlier
layers require only
Gradients from the
pervious layer

$$\frac{\partial \mathcal{L}}{\partial w_{1,1}} = x_1 \frac{\partial \mathcal{L}}{\partial w_1}$$

Backpropagation



Forward Pass

```
y_pred = model(x)
```

Loss Computation

```
loss = MSE(y_pred, y)
```

Backward Pass

```
loss.backward()
```

Update Gradients

```
optimizer.step()
```